

Verifying Efficient Checkpointing Protocol for LLM Training

Qinyu Xu, Jinkun Geng, Joseph Tassarotti, Anirudh Sivaraman

New York University

{qx774, jg7196, jt4767, anirudh}@nyu.edu

(Work-in-Progress Paper)

Abstract

Large-scale LLM training jobs run across thousands of accelerators for weeks, and link flaps, hardware faults, and software crashes interrupt them regularly. To avoid restarting from scratch, a job periodically checkpoints its state and resumes from the most recent snapshot after a failure. Frequent snapshots bound the work lost to a failure, but each one competes with training for time, so recent systems strive to overlap snapshotting with training. Doing so correctly is hard: training state is spread across servers and across the messages in flight between them, and a snapshot assembled from servers that record at different moments may not correspond to any state the job ever passed through. Such a checkpoint undermines the fault tolerance it was meant to provide, since a job restored from it can silently compute wrong results, and the rare interleavings that cause this are beyond what testing can expose.

We present vCheck, a checkpointing protocol for distributed LLM training that is both efficient and formally verified. vCheck adapts the Chandy-Lamport snapshot algorithm to record checkpoints concurrently with training. Its proof, mechanized in Rocq, guarantees that every checkpoint vCheck produces is a globally consistent cut of the execution, a state the job could genuinely have reached, so restoring from it is sound. We have planned to compare vCheck and the conventional checkpointing protocol used by PyTorch. To our knowledge, vCheck is the first checkpointing implementation for LLM training with a machine-checked proof of correctness. vCheck will be open-sourced.

1 Introduction

Training a modern large language model (LLM) enlists tens of thousands of accelerators that must run in concert for weeks or months [17]. At this scale, failures are the common case, not the exception. Link flaps, GPU and memory faults, and node crashes routinely interrupt long-running jobs: Meta reports 419 unexpected interruptions over a single 54-day run of Llama 3 [12], and other operators report comparable rates [16, 17]. Each interruption discards all computation since the last saved state, turning lost progress into wasted GPU time worth millions of dollars on a single run [1]. *Checkpointing*, which periodically persists a consistent snapshot so a job can resume rather than restart, is the primary defense, and a growing body of work has steadily driven its overhead down [13, 14, 23, 30, 31, 33–35].

Checkpoint overhead matters because every checkpoint-

ing protocol balances checkpoint frequency against training speed. Checkpointing more frequently bounds how much progress will lose after failure, but each checkpoint adds work to the training loop and prolongs the run. Checkpointing less often keeps the run fast, but a failure then rolls the job back to a checkpoint far behind the point of the crash, and all progress in between is lost. Conventional implementations, such as PyTorch’s `torch.save`, make this trade-off especially harsh: they write the checkpoint to persistent storage synchronously and stall the critical path of training for the entire write. To reach a better sweet spot, Recent works such as Checkmate [1] and PyTorch’s experimental feature (`torch.async_save`) [24], decouple checkpointing from the training loop, so that persisting a checkpoint overlaps with computation instead of blocking it. This decoupling buys concurrency, but concurrency is also where bugs can easily hide. In §2 we will show, today’s checkpointing solutions contain numerous bugs that break the correctness guarantee a checkpoint exists to provide, which is a serious concern that even outweighs raw performance.

A checkpointing protocol is only useful if the snapshot it produces is correct. LLM training is a highly concurrent computation that interleaves data, tensor, pipeline, and expert parallelism with collective communication such as ring all-reduce and all-to-all. Capturing a *globally consistent* snapshot of such a system asynchronously, without pausing training, is subtle: a silently inconsistent snapshot is worse than no snapshot at all, because a job that resumes from it may diverge or corrupt its results long after recovery. Yet existing high-performance systems optimize aggressively for speed [13, 23, 24, 31] while resting correctness on testing alone, which cannot rule out the concurrency corner cases those optimizations introduce.

Realizing the drawbacks of existing checkpointing implementations, we aim to develop vCheck, an efficient checkpointing protocol with a machine-checked proof of correctness, and the first formally verified checkpointing implementation for LLM training. It guarantees that every snapshot is a globally consistent cut, i.e., a state that could have arisen at a single point of the run across all servers, so resuming from it faithfully continues the training job.

Main challenges. Building vCheck requires addressing two challenges: (1) *How to checkpoint efficiently?* (2) *How to checkpoint correctly?*

How to checkpoint efficiently? Conventional approach (e.g., `torch.save`) sequentially run saving checkpoint and training computation. This is simple but unacceptable for LLM

training, because the checkpoint persistence is expensive and training would be much blocked if the persistence stays on the critical path. To improve the efficiency, vCheck must instead capture the snapshot asynchronously, overlapping it with on-going training so that steady-state throughput is essentially unaffected.

How to checkpoint correctly? Ruling out the bugs that lurk in optimized checkpointing code means reasoning about every possible interleaving of concurrent training and snapshot messages. Testing and model checking are insufficient here, because they cannot cover the unbounded space of executions a real deployment exhibits [7]. Machine-checked formal verification closes this gap. However, formal verification is known to be complex and challenging. Although it has reached file systems, storage, and consensus [3, 8, 15], no prior checkpointing system has been verified in this way.

Key Ideas. We build vCheck by taking the classical Chandy-Lamport snapshot algorithm [6] as a starting point. The Chandy-Lamport algorithm records a *global cut* of a running distributed system. It does so without ever pausing the system. The algorithm circulates *marker* messages. Each marker splits a channel’s stream into the messages that precede the snapshot and those that follow it. This non-blocking behavior is what makes the algorithm attractive for LLM checkpointing. It is also well proven in the role: the Chandy-Lamport algorithm and its descendants underpin the asynchronous snapshots of stream-processing engines such as Apache Flink [2] and graph-processing systems such as GraphLab [21]. However, today’s LLM checkpointing systems take a different path. They either save and recover each server’s state in isolation [10, 13, 23], or they quiesce training behind a global barrier before dumping state [25, 30, 31].

The Chandy-Lamport algorithm alone, however, is not a checkpointing implementation. It is stated over an abstract distributed system in which a process’s state and a channel’s messages are opaque. It dictates *when* to record state and guarantees a consistent cut, but does not define *what* that state is for a training job or how to capture it quickly. An implementation must also handle what the algorithm abstracts away: the coordination and synchronization among threads on one machine, and the communication among machines. vCheck must therefore capture this implementation logic in a formal specification and prove that its implementation satisfies the specification, so that subtle bugs, such as the concurrency bugs already found in PyTorch’s checkpoint implementation (§2), cannot arise.

Our contributions. vCheck will contribute in three ways.

(1) vCheck implements efficient checkpointing by completely decouple the checkpointing logic from the training workflow. Compared with the conventional approach (e.g., `torch.save`), vCheck avoids stalling the training job during checkpointing. Compared with recent approach (e.g., Checkmate [1]), vCheck does not require special hardware (e.g., programmable switches) support and can be deployed in more

general settings.

(2) vCheck builds its proof atop Grove and targets a single theorem, *exact global recovery*, via an invariant that closes every path to concurrency bugs such as torn reads and premature slot reuse. vCheck’s Go implementation is translated into Rocq statement by statement, and we verify the translated code against the model, so the theorem holds of the code that actually runs rather than of an abstraction of it.

(3) vCheck is the first checkpointing implementation backed by a machine-checked proof of correctness. Every snapshot it records is provably a globally consistent cut of the execution. This delivers a level of assurance that testing alone cannot reach.

2 Background and Motivation

Decoupling design for efficient checkpointing implementation. An efficient checkpointing implementation is pivotal to both training speed and progress preservation. With a costly implementation, frequent checkpointing imposes substantial overhead on training and delays its completion. Infrequent checkpointing, on the other hand, loses more progress whenever the training framework crashes and must recover from the latest checkpoint. PyTorch illustrates the cost. Its conventional checkpointing routine, `torch.save`, writes the model state to persistent storage (e.g., disk) synchronously, blocking the training workflow for the entire duration of the write.

To address this inefficiency, recent work has recognized that checkpointing logic should not be tightly coupled to the training workflow. Instead, decoupling has emerged as the guiding principle for building more efficient checkpointing protocols. Gemini [31], DataStates-LLM [22], and Checkmate [1] all follow this principle. The PyTorch community has moved in the same direction: its recent releases integrate the experimental `torch.async_save` feature, which decouples checkpointing from the training workflow [24].

The benefit of decoupling is evident. The cost of checkpointing is hidden behind the training workflow, so a decoupled design incurs far less overhead and far less stall time than a coupled design that executes checkpointing and training computation in sequence. This benefit, however, does not come for free. The conventional coupled design is inefficient, but by running checkpointing and training one after the other it sidesteps the complexity of concurrency altogether. Decoupling buys its efficiency precisely by introducing concurrency, and concurrency brings correctness risk. Compounding the risk, decoupled designs are still new: they have not been rigorously validated, and most implementations remain research prototypes or experimental features. Indeed, by scrutinizing the open-source implementation of `torch.async_save` in PyTorch, we find it to be bug-prone.

Correctness risks in decoupled checkpointing implementations. Recent experimental feature (`torch.async_save`) removes much overheads due to synchronization, but it still

Defect	What happens
Crash mid-save <i>consistency</i>	Filenames carry rank and counter, never the step. One rank writes step $N+1$, another dies first; save raises, so the operator restores what looks like the surviving checkpoint and loads a model half N , half $N+1$.
Reused staging buffer <i>consistency</i>	Buffers are keyed by source storage, so step $N+1$ stages into the buffer step N is still uploading from.
Dead writer thread <i>completeness</i>	An IO error kills a writer; <code>join()</code> returns normally and nothing compares files written against files requested. <code>save</code> succeeds, <code>load</code> fails.
Round with no manifest <i>completeness</i>	Commit returns with no index and no expected count; <code>load</code> rebuilds each shape from the files present. A finished and a half-finished save are indistinguishable.

Table 1: Defects in PyTorch’s asynchronous checkpointing (`torch.async_save`) and the checkpoint property each one violates.

incurs blocking when checkpoint state is generated at high frequency but the saving is bottlenecked by the storage. More importantly, PyTorch assumes only 1 checkpoint can exist in the same time. When there are multiple checkpoint in-flight (e.g., one checkpoint is being persisted whereas another checkpoint is already generated in memory), PyTorch does not have built-in mechanism to manage such concurrency and relies on the user to handle that [24], which leaves more bug-prone risk to the developer. Table 1 summarizes the defects we found. At a high level, all these bugs are related to the concurrency involved in the training framework where checkpointing and training logic are running asynchronously in parallel. Handling such concurrency is always a challenge to the developers since one careless overlook can lead to a subtle bug that is hidden inside the codebase and may only be revealed after long-term laborious testing.

Formal verification as a fundamental approach to ruling out bugs. The defects in Table 1 are not performance blemishes. Each one can hand a resumed job a state that the original run never passed through, so the job silently trains toward a wrong result. Because that state never existed, the run cannot be reproduced either, even though reproducibility is essential in settings such as model tuning and algorithm analysis. Such bugs, however, can hardly be detected, let alone eliminated, by bounded testing: a test exercises only the interleavings it happens to trigger, and every defect in the table lives in a crash or race that ordinary runs rarely hit. Eliminating them requires an argument that covers every interleaving at once, and that is

precisely what formal verification provides. It establishes the correctness of an implementation through a rigorous proof over all possible executions, and having a machine check that proof closes the last gap, since no case can be skipped and no step taken on faith. Formal verification is therefore a promising path to correct and resilient LLM training systems.

Our motivation. Recent literature has invested extensive effort in designing and implementing efficient checkpointing protocols, yet prior work has left the verification of their correctness unaddressed. Given the numerous bugs that keep being reported, correctness has become as important a requirement for today’s checkpoint implementations as efficiency. This gap motivates vCheck. Like prior work, vCheck aims for highly efficient checkpointing that supports fault tolerance in LLM training. Unlike prior work, and more importantly, it also establishes high assurance of correctness: its implementation is rigorously checked by a proof assistant, leaving no room for hidden bugs.

3 System Design

3.1 Overview

vCheck decouples checkpointing from training, and replication from iteration boundaries. Despite this decoupling, the cluster maintains its global awareness: every checkpointer can independently determine which versions are globally complete.

Figure 1 illustrates vCheck’s five-step workflow. The workflow involves two roles: the *trainer*, which runs the LLM training workload, and the *checkpointer*, which saves and replicates the training state. vCheck deploys one checkpointer on each machine, and the checkpointers cooperate to replicate multiple *versions* of the trainers’ state.

Definitions. We describe the workflow based on the following definitions.

Version: a complete and consistent checkpoint of the whole system, from which all trainers can be recovered.

Partition: the portion of a version generated by one trainer. A version is partitioned across machines, and each partition is handled by the checkpointer on the same machine as the trainer that produced it.

Piece: the unit of transmission. When a checkpointer replicates a partition, it splits the partition into pieces and transmits them according to a predefined schedule.

Bank: a memory buffer that each checkpointer pre-allocates to hold partitions, both those produced by its local trainer and those replicated from other checkpointers. A bank is divided into *slots*, each holding one partition.

The five steps proceed as follows.

① **Trainer: Announcing a partition.** After every k training iterations, the trainer announces its newly computed partition to its local checkpointer by passing a descriptor that holds the partition’s memory location, size, and digest.

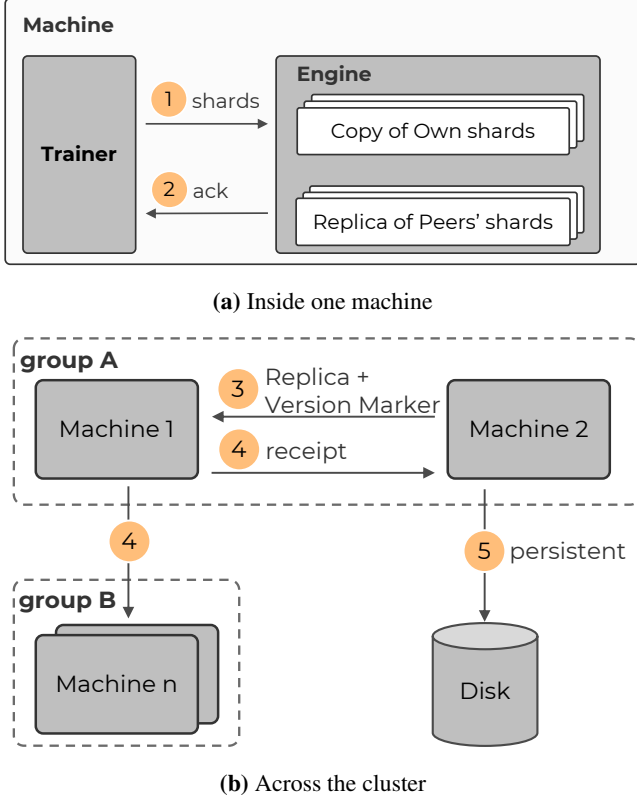


Figure 1: vCheck’s main steps. (a) A version enters the checkpoint (①, ②). (b) It is replicated to its holders (③), acknowledged by receipts (④), and written to disk (⑤). Each checkpoint keeps a copy of its own shard and replicas of its peers’.

② **Checkpoint: Copying and verifying.** The checkpoint copies the memory region into a slot of its own bank and verifies the copy against the digest. Once the copy is complete and verified, it notifies the trainer, which may then reclaim the region.

③ **Checkpoint: Replicating to peers.** The checkpoint streams the partition to the m peer checkpoints assigned to replicate it. Depending on the scheduling policy, the partition may be split into pieces and transmitted over multiple rounds rather than in a single message. When the transmission ends, the checkpoint appends an end-of-version marker to the stream on each channel, which tells the peer that the partition is complete.

④ **Checkpoint: Gossiping partition receipts.** Once a checkpoint receives the marker from a peer, it broadcasts a receipt to all checkpoints, announcing that it holds the partition. By collecting receipts, every checkpoint learns which partitions have been sufficiently replicated. Such a partition is *stable*, and a version is stable once all of its partitions are; each checkpoint tracks the newest stable version as `stable()`. A stable version can restore the system state after a crash.

⑤ **Checkpoint: Dumping to disk.** Every d training iterations, the checkpoint writes the stable partitions it holds to local disk. This persistence guards against extreme failures, such as a cluster-wide power loss that wipes every replica in memory: after such a failure, the system can still recover from the most recently persisted version on disk.

3.2 Memory Copy and Replication ①–③

Steps ① and ② keep checkpointing logic entirely out of the training loop. The trainer passes only a descriptor, which records the memory location, size, and digest of its newly computed partition. The checkpoint then reads and copies the partition into its bank without blocking the trainer’s computation.

To prevent torn reads, vCheck treats the descriptor as a borrow of the region: the checkpoint holds read-only access, and the trainer must not reclaim the region until the checkpoint returns it. The borrow is the only synchronization point between the two roles. Users can configure the trainer either to wait for the return or to drop the pending checkpoint and keep training without interruption. Crucially, even in the blocking mode, the trainer waits only for a local memory copy, never for the network.

To guarantee data integrity without further synchronization, the checkpoint validates the copied bytes against the digest in the descriptor. Any mismatched partition is discarded, so corrupted data never enters the replication pipeline and the checkpoint’s copy always matches the trainer’s output exactly. Furthermore, vCheck handles the validated partition with no intermediate buffering: the checkpoint copies the bytes directly into a slot of its bank and uses that same slot both to stream the partition to peers for replication.

3.3 Gossip Partition Receipts ④

Because vCheck decouples checkpointing from training, no global barrier stops the trainers while the checkpoints finish replicating a version. A checkpoint may therefore receive pieces that belong to different versions from the same peer. vCheck sends the pieces over in-order network channels, so a channel carries a peer’s pieces version after version, and the end-of-version markers delimit them. Using the markers, the receiver assigns each piece to its version and assembles the pieces into the complete partition for that version.

However, barrier-free replication raises a memory-management challenge. In vCheck implementation, each checkpoint’s bank is a ring buffer of W slots. A fast checkpoint can fill its bank while a slow peer is still receiving an older partition, and if it overwrites a slot prematurely, it may destroy a partition that belongs to the newest globally complete version. Before reusing a slot, a checkpoint must therefore confirm that a newer, globally complete version exists across all peers, i.e., the new version is already stable. vCheck establishes this through gossip, by which checkpoint-

ers share their replication progress.

Once a checkpointer has received a partition and its end-of-version marker, it broadcasts a receipt to all other checkpointers. From the gossiped receipts, each checkpointer derives which versions have been fully replicated and marks them as stable, advancing a `stable()` watermark that records the newest stable version. A checkpointer overwrites a bank slot only if the partition in that slot belongs to a version below the watermark. Otherwise, it blocks until the watermark advances past that version.

3.4 Dumping to Disk ⑤

To survive catastrophic failures in which every checkpointer replicating the partition has crashed, vCheck maintains a second tier of persistence on disk. Traditional disk checkpoints require cluster-wide coordination: each rank writes its state, a global barrier confirms that all writes have succeeded, and only then is the checkpoint declared valid. vCheck instead decouples disk I/O from cluster coordination entirely. Checkpointers independently write the partitions they hold to local storage, one standalone file per partition, consisting of a header that records the partition identifier, the version number, and the length, followed by the raw bytes.

The disk path follows a copy-then-write pattern. To write a file, the checkpointer pins the corresponding bank slot and copies its contents into a dedicated buffer reserved for disk checkpoints. A separate writer thread then streams the buffer to disk in parallel. While the write is in progress, the trainer, the network transport, and the gossip mechanism all continue without interruption.

3.5 Recovery from Checkpoints

To recover the system state based on checkpoint, vCheck takes two steps. The protocol first attempts a fast recovery from memory: it queries every reachable checkpointer for a target version t , and each checkpointer replies with the validated partitions it holds for t together with its local `stable()` watermark. If t is older than a checkpointer’s watermark, the checkpointer rejects the request and returns its latest stable version, then the protocol restarts the query at that higher version. Once the replies are collected, the protocol assembles a complete version from one replica of each partition.

If the entire cluster has crashed and all checkpoints have been erased from memory, the protocol falls back to the disk tier. It scans the storage directory, discards every file whose physical length disagrees with its header, and restores the newest version for which a valid file exists for every partition.

4 Formal Verification Strategy

A checkpointer in vCheck reads each shard directly out of trainer memory and replicates it to peer checkpointers without a global barrier. Both the direct read and the barrier-free

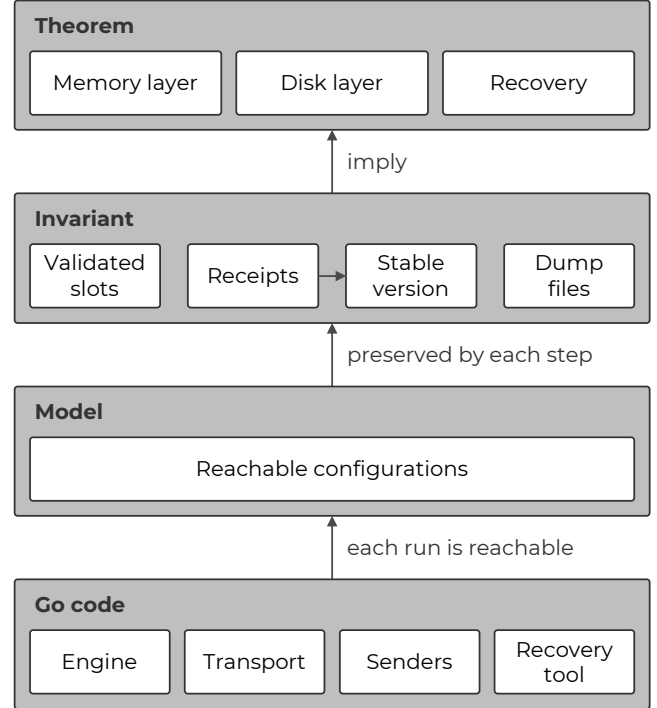


Figure 2: Structure of the proof. Every run of the Go code ends in a reachable configuration of the model (§4.2). Every step preserves the invariant, and four of its clauses imply the theorem (§4.3). The recovery clause also uses the recovery tool’s verified assembly.

replication open a window for a silent error. The first is a *torn read*: the checkpointer copies a shard while the trainer is still overwriting it for the next iteration, so the copy mixes old bytes with new ones and matches no state that the trainer ever produced. The second is *premature slot reuse*: a checkpointer overwrites its copy of a version that recovery still needs. Either error yields a checkpoint that looks complete but is not. We prove in Rocq that neither can happen. Call a set of shards a *correct checkpoint* of version v if it contains every shard of the model and each shard is byte for byte what the trainer produced at version v . Our theorem, **exact global recovery**, gives one guarantee per tier. In memory, as long as every shard keeps a live holder, the live checkpointers as a whole hold a correct checkpoint of some version at least as new as any `stable()` that a live checkpointer reports. On disk, whenever every shard has a dump file for a version, those files form a correct checkpoint of that version. Recovery therefore restores a correct checkpoint from either tier. When it restores from memory, the version is no older than any `stable()` in the replies it used.

Figure 2 shows the structure of the proof, which has two halves. The first half is purely mathematical. We describe the whole cluster as a state machine. Each of its states, which we call a *configuration*, records what every checkpointer holds and what every channel has carried. A handful of steps move

the state machine from one configuration to another. We state the theorem over this state machine (§4.1). To prove it, we identify an *invariant*: a property of configurations that holds at the start and that every step preserves. Every configuration the state machine can reach therefore satisfies the invariant, and we show that the invariant implies the theorem (§4.3). The second half connects the state machine to the Go code. The code knows nothing of configurations; each checkpoint holds only its own Go data structures. We therefore define how the data structures of all checkpoints, together with the messages in flight, map to a configuration. We then prove that every operation that changes a checkpoint’s state moves this configuration by exactly one step of the state machine. A run of the code is thus a sequence of steps, so it ends in a configuration that the state machine can reach, and the theorem applies to it (§4.2). This half is carried out in Grove [27], whose Goose front end [4] translates Go into a form that Rocq can reason about. The proof rests on six assumptions, which §4.4 lists.

4.1 Formal Model and Theorem

Modeling the entire system, however, is hard. The system includes the trainer, which is tens of thousands of lines of code and changes constantly. Luckily, the decoupling design of §3 keeps the trainer out of the checkpoint’s state. The two share nothing but a descriptor. We thus model the trainer only by its output: $oracle(s, v)$ denotes the bytes the trainer produced for shard s at version v , and the descriptor’s digest is the digest of these bytes. Therefore, the model covers only the checkpoints and the channels between them. A configuration records only what each checkpoint holds and what each channel carries, and the steps are only the checkpoints’ actions.

For each checkpoint, a configuration records its bank, its receipt matrix, and its disk marks. The bank is a set of slots. A slot holds one shard at one version, and a bit says whether the checkpoint has validated it. The receipt matrix holds the receipts the checkpoint has recorded, and $stable()$ is computed from it. The disk marks say, for each slot, whether it is pinned for a disk write and whether it has been written. For each channel, a configuration records every message sent on it, in order, and how many of them the receiver has taken.

The model includes five steps to capture every change to the system’s state. Each is one operation that a checkpoint performs while it holds its lock. **COPY** copies the shard that the trainer announced into a slot of the bank, and validates the slot if the digest of the copied bytes matches the digest in the descriptor. **SEND** appends one message to a channel. A message carries shard data, an end-of-version marker, or gossip. **RECEIVE** takes the next message from a channel and applies it: shard data goes into a slot, a marker validates the slot that it completes, and gossip goes into the receipt matrix. **PIN** pins a slot before the checkpoint writes it to

disk. **UNPIN** releases the slot after the write and marks it as written.

To support flexible transmission schedules, the model does not fix how **SEND** splits a version into messages or when it sends them. Any schedule is covered by the theorem, so a new schedule needs no new proof.

The theorem concerns only configurations that a real run can produce. We call a configuration *reachable* if finitely many steps lead to it from the empty configuration.

Theorem (exact global recovery). Let C be a reachable configuration, and let L be any set of checkpoints that survive a failure.

Memory layer. Given that every shard has one of its assigned holders in L , and that some checkpoint in L records $stable() = t > 0$, some version $T \geq t$ is complete and consistent in L . Complete: every shard s has a validated copy of (s, T) at a holder in L . Consistent: every validated copy of (s, T) at any checkpoint in L equals $oracle(s, T)$.

Disk layer. Every dump file that a checkpoint wrote for shard s at version v holds $oracle(s, v)$.

Recovery. Given that the recovery tool assembles version t from the replies of checkpoints in L and from dump files, every shard of the assembled model equals $oracle(s, t)$, and t is at least the $stable()$ in every reply it used.

4.2 Connecting the Model to the Code

The theorem is stated about the state machine, not the code. To bridge the two, we map the code’s state to a configuration and show that this configuration is reachable under any order of operations. Thus, we verify that each operation a checkpoint performs maps to exactly one model step. The proof keeps a log of the steps taken so far, and by induction over the execution the mapped configuration is always reachable. In this way, the theorem holds for the code. For **COPY**, **PIN**, and **UNPIN**, this is direct, because only the checkpoint’s local state is concerned. However, a send or a receive also concerns the channel between two checkpoints, which makes the mapping harder.

Receives. The model’s **RECEIVE** takes the next message on the channel. But in a real network, messages may be dropped, duplicated, or reordered. We prove that the transport delivers each channel’s messages in the order they were sent, with none skipped or repeated, so every receive in the code is a **RECEIVE** in the model.

Sends. In the model, **SEND** appends a message to a channel, but it does not accept every message. It requires four things of the sender. Versions go out in order on each channel, or late data for an old version could land in a slot already reused. All of a version’s data goes out before its marker, or the receiver would validate a half-filled replica. Data comes from a validated slot, or the receiver would validate bytes the trainer never produced. Gossip reports only receipts the sender has recorded, or $stable()$ would rise above the newest complete

version and slots would be reused too early. The schedule is free, but every message must meet these requirements. The sender is free code, so nothing enforces them by construction. The proof therefore keeps a record of what each channel has carried so far and checks each new message against it and against the sender’s own state, so every send in the code is a SEND in the model.

4.3 The Core Safety Invariant

Since a reachable configuration is produced by finitely many steps, we prove the theorem by induction over steps. The theorem itself, however, is not inductive. It describes what a configuration holds, but a step preserves that only because of how the checkpoints got there, which the theorem does not state. We therefore prove an inductive property instead, the invariant. It holds in the empty configuration, each of the five steps preserves it, and it implies the theorem, so every reachable configuration satisfies both. It has four clauses, each closing one path to one of the two failures named at the start of this section.

Validated bytes are the trainer’s. A slot is validated either by COPY or by RECEIVE. COPY validates only when the copied bytes match the descriptor’s digest. That digest is the digest of the trainer’s bytes (assumption (1)), and a torn read would break the match (assumption (2)). RECEIVE validates only when the marker arrives, and the requirements on SEND ensure that everything before the marker came from a validated slot. Either way, the validated bytes are the trainer’s.

Receipts are honest. A holder sends a receipt for (s, v) only after it has validated its copy of (s, v) , and gossip carries only receipts that the sender has recorded. So a receipt in any receipt matrix means that its holder validated that copy.

The newest stable version is never overwritten. Let T be the largest `stable()` that any checkpoint has recorded. `stable()` reaches T only when the matrix holds a receipt for T from every holder of every shard, so by the previous clause every holder has validated its copy of T . A holder reuses a slot only for a version below its own `stable()`, which is at most T . So no holder ever reuses its copy of T .

Dump files hold validated bytes. PIN pins only a validated slot, no step reuses a pinned slot, and the checkpoint writes the dump file while the slot is pinned. So every dump file holds the bytes of a validated slot, for the shard and version it was written for.

The memory layer follows from the first and third clauses: every holder in L keeps a validated copy of $T \geq t$, and every validated copy is the trainer’s bytes. The disk layer follows from the first and fourth. Recovery follows from the same two, because a checkpoint’s reply contains only validated copies and the tool uses only complete dump files, and the tool asks again whenever a reply reports a `stable()` above t , which gives the bound on t .

Layer	Lines
Code proofs	22,768
Transport engine	3,213
Model, invariant, and preservation	28,347
Protocol mathematics	254
Trace coupling and recovery	2,726
Top-level claims and closed run	2,316
Total	60,926
Proof-to-code ratio	16.3×

Table 2: Lines of proof by layer, excluding blanks and comments, and the ratio of total proof lines to the 3,742 lines of Go.

4.4 Trusted Assumptions

The theorem holds of a running cluster under six assumptions. (1) On the trainer side, the memory copy returns the bytes of the announced region, and each descriptor for a version newer than every earlier one carries the digest of $oracle(s, v)$. COPY ignores older or repeated descriptors, so nothing is assumed about them. (2) The digest function, FNV-1a-128, has no accidental collision within a run. We do not consider adversarial inputs. (3) Grove’s network and file primitives meet their specifications. (4) Every checkpoint starts with the same flags and runs once. (5) The recovery tool’s sockets deliver its requests and replies intact. (6) The Rocq kernel, Iris, Perennial, and Goose are sound.

5 Proof Effort

vCheck’s verified checkpoint is 2,173 lines of Go. The trusted code around it, namely the copy primitive, the control channel, the two daemons, and the recovery tool, is another 1,569 lines of Go, plus a 990-line PyTorch trainer. Table 2 breaks down the proof, in lines of Rocq, by layer.

The first two rows, which prove the Go code against the model, amount to about 16.3 lines of proof per line of code, within the range reported by Grove [27], vMVCC [7], and GoJournal [5].

6 Planned Evaluation

Because of its decoupled design, vCheck is implemented as a pluggable module that works with existing training frameworks with minimal integration effort. We are currently connecting vCheck to PyTorch through shared memory, so that vCheck can checkpoint and replicate the state of every node in an LLM training job. We then plan to compare vCheck against PyTorch’s built-in checkpointing mechanism and measure how each implementation affects end-to-end training time.

```

if kind == wire.KindChunk {
    src, ver, idx, data, ok := wire.DecodeChunk(m)
    if ok == false { return st }
    out := store.Chunk(st, src, ver, idx, data) // buggy
    // out, filed := store.Chunk(st, src, ver, idx, data, floor)
    // if filed == false { return out }
    tk, _, _ := tracker.NoteChunk(trackerOf(out), src, ver, idx)
    return replaceTracker(out, tk)
}

```

Listing 1: The receive transition before the fix; the commented lines are the fix.

7 Learned Lessons

We structured vCheck so that the reasons for its correctness are visible in the code, and we tested it throughout with unit tests, a cluster simulator, and end-to-end runs that kill processes and compare recovered bytes against the trainer’s. Verification still found three bugs that the tests did not reach, and it turned two policies into rules.

A refusal must be visible to the caller. Listing 1 shows the RECEIVE transition before the fix. The store may refuse a data message: its slot may hold a version that must be kept, the message may arrive out of order, or it may overrun the slot. The store signaled a refusal by returning the state unchanged, and the transition then recorded the message as received. A holder could thus issue a receipt for bytes it did not hold. Once every holder had done so, `stable()` would certify a version that no one could restore. No test reached this path, because the store refuses only after a sender runs W versions ahead of a holder. The proof found the bug when we stated the second clause of §4.3, which the code could not satisfy. The fix makes the store report whether it accepted the message.

Specify every primitive as one that may fail. When the copy primitive failed partway, the partial bytes stayed in a slot that the checkpointer had already claimed. Proving COPY against a primitive that may fail exposed the bug. A failed copy now leaves the slot untouched.

Exact bytes can still be stale. The recovery tool checked only at its first request that its proposed version was at or above every survivor’s `stable()`. A survivor whose `stable()` advanced in between refused the request, and the tool assembled the version from the other replies. The bytes were exact, but the version was older than one the cluster had certified. The recovery clause of the theorem requires t to be at least the `stable()` in every reply that recovery used. Its proof did not go through until the tool asked again at the higher version.

The proof turns policies into rules. Reusing validated slots only below `stable()` was the simulator’s policy until the third clause of §4.3 required it of the checkpointer. Recovery asks the checkpointers for their shards rather than reading their memory, because a theorem over the survivors’ memory had a premise that no code produced.

8 Related Work

Fault-tolerant LLM training. Large-scale LLM training must tolerate a variety of failures in practical deployments. Existing work falls into two classes: checkpoint-based and checkpoint-free approaches. Checkpoint-free approaches tolerate failures by exploiting the redundancy that already exists in the training job. Bamboo [29], for instance, has each node in a pipeline redundantly compute the layers of its neighbor, so that when a node is preempted its neighbor already holds the state needed to carry on. ReCycle [11] exploits data parallelism, in which every pipeline stage has peers holding identical parameters. When a node fails, ReCycle reroutes its micro-batches to those peers, fitting the rerouted work into existing pipeline gaps rather than stalling the pipeline. FleetIQ [9] exploits the rich connectivity of LLM training networks. When a NIC or a link flaps, it reroutes traffic over healthy links and NICs, masking the failure from the training framework above. Checkpoint-free approaches are attractive, but each applies only to specific workloads, failure types, or deployment settings. Fault tolerance for general large-scale LLM training therefore still requires checkpoint-based approaches, which have drawn numerous efforts in the recent literature, as we describe next.

Efficient checkpointing implementation. Prior work reduces the overhead of checkpointing so that checkpoints can be taken frequently. One line overlaps persistence with training. CheckFreq [23] pipelines the snapshot of model state with the next iteration and tunes the checkpoint frequency online. DataStates-LLM [22] exploits the fact that model and optimizer shards remain immutable for long stretches of an iteration and lazily copies them to host memory, and then to storage, in the background. PyTorch’s asynchronous distributed checkpointing adopts the same staging pattern [24], and ByteCheckpoint [30] adds a parallelism-agnostic checkpoint format with load-time resharding and full-stack I/O optimizations. A second line keeps checkpoints in memory and relies on replication to tolerate crash failures. Gemini [31], PHOENIX [34], Swift [35], and Just-In-Time Checkpointing [13] all keep each node’s state in the memory of its peers, either by replicating it explicitly or by exploiting the replicas that data parallelism already maintains, and fetch that state to recover when a node crashes. Finally, Checkmate [1] leverages programmable switches to mirror gradients in flight to shadow servers, which apply them to reconstruct the checkpoint state. Checkmate adds negligible overhead to training, but it depends on special hardware and is therefore limited in where it can be deployed. vCheck’s implementation is inspired by these works and incorporates both the decoupling design and the replication approach to checkpoint efficiently. Unlike prior work, however, vCheck aims for both high performance and high fidelity in its checkpointing implementation. Whereas all prior systems optimize for speed and validate their designs by testing alone, vCheck establishes its correct-

ness with a machine-checked formal proof.

Formal verification framework. vCheck builds on a line of foundational verification frameworks developed in Rocq (formerly Coq) [28]. Iris [18] provides a higher-order concurrent separation logic. Perennial [3] extends Iris with crash-safety reasoning and, through Goose [4], applies it to Go programs. Grove [26, 27] further adds distributed-systems primitives such as RPCs, replicated state machines, and leases. We build vCheck on Grove for its mature separation-logic support and rich tactic library. vCheck’s proof methodology, however, is not tied to Grove. In principle, it applies equally to other verification frameworks, including Dafny [20], IronFleet [15], Verdi [32], and Verus [19].

9 Conclusion

We have presented vCheck, an efficient checkpointing protocol that provides fault tolerance for LLM training without slowing it down. vCheck decouples checkpointing from the training workflow, so training never blocks on a checkpoint in progress. Building on Grove, we develop a machine-checked proof for vCheck, to demonstrate that every checkpoint it produces is a state the training job could genuinely have reached, so resuming from it is sound. vCheck thus offers both high efficiency and high fidelity in checkpointing. We plan to evaluate vCheck against conventional checkpointing implementations in the near future, and we will open-source both its implementation and its proof.

References

- [1] Ankit Bhardwaj, Weiyang Wang, Jeremy Carin, Adam Belay, and Manya Ghobadi. Checkmate: Zero performance overhead model checkpointing via network gradient replication. In *NSDI*, 2026.
- [2] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. State management in apache flink: consistent stateful distributed stream processing. In *VLDB*, 2017.
- [3] Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying concurrent, crash-safe systems with perennial. In *SOSP*, 2019.
- [4] Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying concurrent Go code in Coq with Goose. In *CoqPL*, 2020.
- [5] Tej Chajed, Joseph Tassarotti, Mark Theng, Ralf Jung, M. Frans Kaashoek, and Nickolai Zeldovich. GoJournal: a verified, concurrent, crash-safe journaling system. In *OSDI*, 2021.
- [6] K. Mani Chandy and Leslie Lamport. Distributed snapshots: determining global states of distributed systems. *ACM Trans. Comput. Syst.*, 1985.
- [7] Yun-Sheng Chang, Ralf Jung, Upamanyu Sharma, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying vMVCC, a high-performance transaction library using multi-version concurrency control. In *OSDI*, 2023.
- [8] Haogang Chen, Tej Chajed, Alex Konradi, Stephanie Wang, Atalay İleri, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying a high-performance crash-safe file system using a tree specification. In *SOSP*, 2017.
- [9] Clockwork. Unflappable fabrics: Ending link-induced chaos in GPU clusters. Clockwork FleetIQ white paper, 2025.
- [10] Clockwork. Torchpass workload fault tolerance, 2026.
- [11] Swapnil Gandhi, Mark Zhao, Athinagoras Skiadopoulos, and Christos Kozyrakis. ReCycle: Resilient training of large DNNs using pipeline adaptation. In *SOSP*, 2024.
- [12] Aaron Grattafiori and et al. The llama 3 herd of models, 2024.
- [13] Tanmaey Gupta, Sanjeev Krishnan, Rituraj Kumar, Abhishek Vijeev, Bhargav Gulavani, Nipun Kwatra, Ramachandran Ramjee, and Muthian Sivathanu. Just-in-time checkpointing: Low cost error recovery from deep learning training failures. In *EuroSys*, 2024.
- [14] Shujie Han, Feng Jiang, Patrick P. C. Lee, Xiao Zhang, Zhijie Huang, Nannan Zhao, Xiaonan Zhao, and Lichen Pan. TierCheck: Tiered checkpointing for fault tolerance in large language model training, 2026.
- [15] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath T. V. Setty, and Brian Zill. Ironfleet: Proving practical distributed systems correct. In *SOSP*, 2015.
- [16] Tao He, Xue Li, Zhibin Wang, Kun Qian, Jingbo Xu, Wenyan Yu, and Jingren Zhou. Unicorn: Economizing self-healing llm training at scale, 2023.
- [17] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *NSDI*, 2024.
- [18] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order

- concurrent separation logic. *Journal of Functional Programming*, 28, 2018.
- [19] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. In *OOPSLA*, 2023.
 - [20] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, 2010.
 - [21] Yucheng Low, Joseph Gonzalez, Aapo Kyrola, Danny Bickson, Carlos Guestrin, and Joseph M. Hellerstein. Distributed GraphLab: A framework for machine learning and data mining in the cloud. In *VLDB*, 2012.
 - [22] Avinash Maurya, Robert Underwood, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. DataStates-LLM: Lazy asynchronous checkpointing for large language models. In *HPDC*, 2024.
 - [23] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. CheckFreq: Frequent, fine-grained DNN checkpointing. In *FAST*, 2021.
 - [24] Lucas Pasqualin, Iris Zhang, Rodrigo Kumpera, Chien-Chin Huang, and Yunsheng Ni. Asynchronous saving with Distributed Checkpoint (DCP). PyTorch Tutorials, 2024.
 - [25] PyTorch. Distributed checkpointing, 2023.
 - [26] Upamanyu Sharma. *Modular Verification of Distributed Systems with Grove*. PhD thesis, Massachusetts Institute of Technology, 2022.
 - [27] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, Frans Kaashoek, and Nickolai Zeldovich. Grove: a separation-logic library for verifying distributed systems. In *SOSP*, 2023.
 - [28] The Rocq Development Team. The Rocq Prover. <https://doi.org/10.5281/zenodo.15149629>, 2025. Version 9.0.
 - [29] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *NSDI*, 2023.
 - [30] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, Xin Liu, and Chuan Wu. ByteCheckpoint: A unified checkpointing system for large foundation model development. In *NSDI*, 2025.
 - [31] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. GEMINI: Fast failure recovery in distributed training with in-memory checkpoints. In *SOSP*, 2023.
 - [32] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. Verdi: A framework for implementing and formally verifying distributed systems. In *PLDI*, 2015.
 - [33] Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. ReFT: Representation finetuning for language models, 2024.
 - [34] Haotian Xie, Junlin Chen, Mingkai Zheng, Lishan Yang, and Zhao Zhang. PHOENIX: Resilient LLM training with hot-swapping via zero-overhead checkpoint, 2026.
 - [35] Yuchen Zhong, Guangming Sheng, Juncheng Liu, Jinhui Yuan, and Chuan Wu. Swift: Expedited failure recovery for large-scale DNN training. In *PPoPP*, 2023.